

NEBULA 2026

Constraint Optimization through RTL Enhancement Using Generative AI



Project by
Team Chipsi

Pranav Pandit
Aditya Vartak
Sathvik S Srikanth

September 2026

Contents

1	Problem Statement	1
2	Key Features of RTL Framework	1
3	RTL Benchmark	2
4	Framework Architecture	3
5	Process of the Framework: The Entire Flow	3
5.1	Initialization	3
5.2	Baseline Characterization	4
5.2.1	Initial Synthesis (Yosys)	4
5.2.2	Initial Timing Analysis (OpenSTA)	4
5.2.3	Report Parsing	4
5.2.4	Establishing the Baseline Metrics	5
5.3	The Iterative Optimization Loop	5
5.3.1	Violation Selection	5
5.3.2	Violation Classification	6
5.3.3	LLM-Driven Patch Generation	8
5.3.4	Patch Validation (Icarus Verilog)	8
5.3.5	Re evaluation	9
5.3.6	Formal Safety Verification (EQY/SymbiYosys)	9
5.3.7	Accept / Reject Decision	10
5.3.8	Iteration Logging	10
5.4	Whole Design Formal Sign off	10
5.5	Physical Design (OpenROAD)	11
6	Reporting and Dashboard	12
6.1	Structured Reports (<code>logger.py</code>)	12
6.2	Dedicated Formal Equivalence Verification Report	12
6.3	The Offline HTML Dashboard (<code>generate_demo.py</code>)	12
7	Findings During Exhaustive Testing of the LLM	14
8	Conclusion	15

1 Problem Statement

Achieving timing closure is one of the most significant challenges in modern digital chip design, requiring engineers to repeatedly analyze timing reports, trace critical paths, and manually restructure RTL to meet performance targets across multiple design iterations. This process is time-consuming and scales poorly with increasing design complexity, multi-clock-domain architectures, and asynchronous interfaces. This project leverages Generative AI to analyze timing bottlenecks and recommend targeted RTL optimizations — pipelining, retiming, logic restructuring, and FSM optimization — thereby accelerating timing closure. Every recommended change is formally verified against the original RTL to ensure functional correctness is preserved throughout the optimization process.

2 Key Features of RTL Framework

- Automated Yosys, OpenSTA, and OpenROAD report parsing and detailed processing. (Added Feature: **Dynamic Script Generation** for Synthesis and Formal Verification)
- **Heuristic classifier** suggesting 6 RTL optimization techniques per violation (pipelining, restructuring, retiming, register balancing, FSM re-encoding, resource duplication)
- **Multi-LLM support**: Tested on Anthropic Claude, Gemini, and OpenAI - configurable
- **Self-correcting prompt loop**: retries on syntax errors and rejected patches with targeted feedback to the LLM
- Formal equivalence verification (EQY) on every accepted patch (**Latency Aware Equivalence Checking**), plus a **whole-design formal sign-off** on the final RTL before PnR is attempted
- **CDC/async-reset safety exclusion**: never auto-patches metastability-sensitive paths and flags them to the user at the end
- **One-shot OpenROAD Physical Design (PnR)**: When enabled, runs PnR and provides synthesis-vs-post-route PPA comparison with **automatic timing sign-flip detection**
- Structured logging with auto-generated Markdown report, **dedicated Formal Equivalence Verification Report**, and PPA comparison report
- **Optional user intervention**: confirm continuation after each iteration and manually resolve ambiguous formal-vs-timing conflicts configurable, defaults to safe auto-reject when disabled
- Self-contained offline HTML dashboard: tabular PPA breakdown, timing convergence charts, decision history, and formal verification status, viewable in any browser with zero setup
- **Scalable and flexible**: Easily configurable and extensively tested across multiple designs
 1. Baseline RISC-V based SoC (5 clocks)
 2. PicoRV32-based SoC (6 async clocks, contains ALU/math accelerators)
 3. Multi-clocked counter design

3 RTL Benchmark

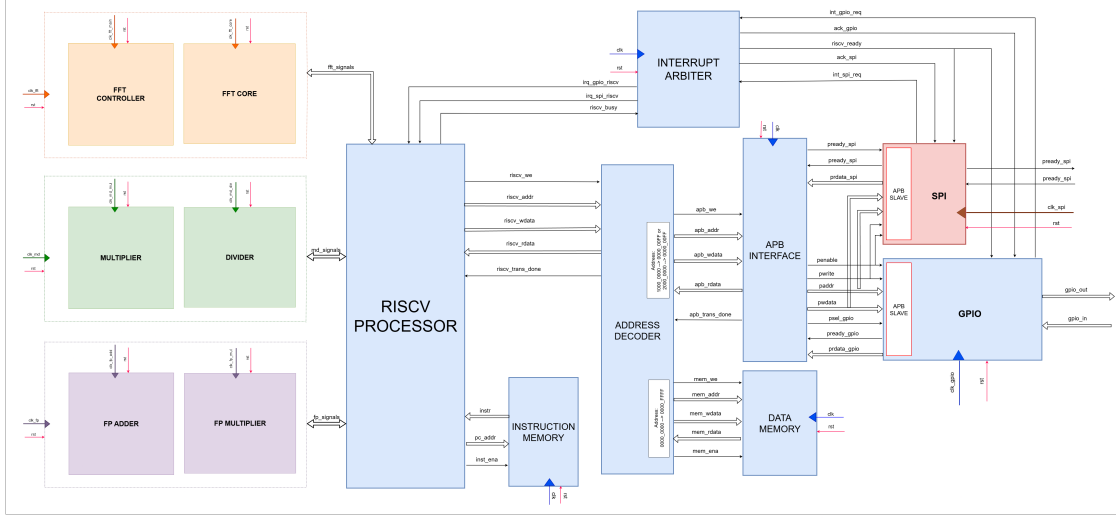


Figure 1: RTL Benchmark: RISC-V based SoC

- **Architecture:** RISC-V SoC with a pipelined core, and SPI and GPIO as memory-mapped peripherals over an APB bus via a dedicated address decoder.
- **Accelerators:** The SoC integrates five hardware accelerator blocks:
 1. 8-point FFT (controller + core)
 2. 32-bit multicycle multiplier
 3. 32-bit divider
 4. Floating-point adder
 5. Floating-point multiplier
- **Clocking:** Each accelerator operates on its own clock. The SoC has five asynchronous clock domains with derived clocks:
 1. `clk` → divided by 2 for GPIO
 2. `clk_md` → divided by 3 for multiplier, divided by 6 for divider
 3. `clk_fp` → divided by 2 for FP adder, divided by 4 for FP multiplier
 4. `clk_fft` → divided by 1 for FFT core, divided by 4 for FFT controller
 5. `clk_spi` → divided by 3 for SPI
- **CDC Handling:** Clock domain crossings are handled using dedicated modules: **Handshake bridge**, **APB CDC bridge**, and **Reset synchronizer**. Domain boundaries are color-coded in the architecture diagram.
- **Design Scale:** Post-synthesis, the design is approximately **52K** standard cells.

4 Framework Architecture

The framework is implemented as a modular Python pipeline, with each module responsible for a single stage of the closed-loop optimization process.

- **config.py** – Central settings: paths, clocks, LLM choice, timeouts, feature toggles.
- **orchestrator.py** – Runs the main loop; decides accept reject; ties everything else together.
- **report_parser.py** – Turns raw STA PnR reports into usable data.
- **module_extractor.py** – Maps a violation to its RTL module and code.
- **prompt_builder.py** – Builds the LLM prompt with technique guesses.
- **llm_client.py** – Calls Claude Gemini GPT and extracts the patch.
- **patch_applier.py** – Writes the patch into a new RTL iteration folder.
- **syntax_check.py** – Runs Icarus Verilog to catch broken syntax early.
- **synth_runner.py** – Runs Yosys synthesis with a per module clock target.
- **sta_runner.py** – Runs STA and pulls back timing reports.
- **formal_check.py** – Runs EQY and SymbiYosys to verify functional equivalence.
- **pnr_runner.py** – Runs OpenROAD PnR once, on the final design.
- **logger.py** – Logs every iteration; builds the markdown and formal reports.
- **generate_demo.py** – Builds the offline HTML dashboard.

5 Process of the Framework: The Entire Flow

5.1 Initialization

Every run begins by loading the framework’s central configuration **config.py**, which defines every parameter the rest of the pipeline depends on: the path to the baseline RTL, the top-level module name, the Liberty timing library, per-tool workspace directories and timeouts, the LLM provider selection, and the full clock-domain mapping including each clock’s period, its true asynchronous family (for cross-domain detection), and which RTL module runs on which clock.

A few important configurations are listed below:

Configuration	Description
TOP_MODULE	Specifies the top level module of the design.
LIBERTY_PATH	Specifies the path to the Liberty timing library used for synthesis and timing analysis.
LLM_PROVIDER	Selects the LLM provider used for RTL optimization (claude , gemini , or openai).

Configuration	Description
PNR_ENABLED	Enables or disables the Physical Design / PnR stage (1 = enabled, 0 = disabled).
OVERALL_FORMAL_CHECK_ENABLED	Enables or disables whole design formal equivalence checking (1 = enabled, 0 = disabled).
INTERACTIVE_MODE	Enables or disables user interaction during the optimization process (1 = enabled, 0 = disabled).
MAX_ITERATIONS	Specifies the maximum number of optimization iterations allowed in a run.

The framework then establishes the original, unmodified RTL as its fixed reference point. This baseline copy is never altered for the remainder of the run every later synthesis, timing, and formal comparison is measured explicitly against it. This is saved as the "baseline" RTL design for that particular run of the framework.

5.2 Baseline Characterization

Before any optimization can begin, the framework must establish exactly where the design currently stands.

5.2.1 Initial Synthesis (Yosys)

`synth_runner.py` runs the first synthesis pass on the untouched baseline.
(Technology Used → **TSMC 65 nm**)

- It includes a **synthesis script automator**, i.e, by taking `MODULE_CLOCK_MAP` and a few other configuration features from the `config.py`, the framework itself dynamically generates the required yosys script for any given RTL as input.
- Modules with no explicit clock mapping (pure combinational glue, clock dividers) are still synthesized correctly through a final catch-all pass targeting `PRIMARY_CLOCK`, ensuring nothing is left in a generic, unmapped state.
- Generates Area Reports for further PPA comparisons

5.2.2 Initial Timing Analysis (OpenSTA)

`sta_runner.py` then runs the hand-written `sta.tcl` (Taken from the user directories) script against this freshly synthesized netlist, in an isolated copy of the workspace so no partial state leaks between runs. This produces the raw OpenSTA timing reports, including per-domain worst slack, total negative slack, and full critical-path breakdowns.
(Technology Used → **TSMC 65 nm**)

5.2.3 Report Parsing

These raw text reports are immediately parsed by `report_parser.py` into structured data. The parsed data is processed to **identify the worst violation** from the timing reports in `critical_paths.txt`. The file also contains objects that capture every stage's **delay and fanout**, along

with **worst slack**, **total negative slack**, and **power**. This same parsing layer will be reused for every iteration going forward, ensuring that the baseline and every later comparison are measured through an identical and consistent lens.

5.2.4 Establishing the Baseline Metrics

From this parsed data, the framework computes the numbers that define success or failure for the rest of the run: baseline WNS and TNS, baseline area and power, and, critically, a **separate achievable frequency** for each of the design’s clock domains. This frequency is computed using each domain’s own worst slack together with its own clock period. This avoids a significant class of errors that the framework is specifically designed to prevent: combining one domain’s period with another domain’s violation can produce a frequency value that does not correspond to any actual clock in the design.

A multi-clock SoC does not have a single “frequency”. The `report_parser.py` captures one actual frequency value for each clock domain in the baseline.

$$f_{\text{achievable}} = \frac{1}{T_{\text{clock}} + \text{Worst Slack}}$$

This baseline snapshot, consisting of WNS, TNS, area, power, and per-domain frequency, is what every later iteration’s improvement or regression is measured against, all the way through to the final PPA comparison.

```
=====
NEBULA - GenAI RTL Timing Optimization Framework
=====
Run ID       : 20260913_132521_9237cf
Baseline RTL : ../verilog
Top module   : soc_top
LLM provider : gemini
Max iters    : 10

Establishing baseline (synthesis + STA)...
Baseline:
WNS (setup): -1.59 ns
TNS (setup): -164.56 ns
Area:       229715.64
Max Frequency by domain:
clk         21.72 MHz (worst slack 3.97 ns @ 50.0 ns period)
clk_md      263.85 MHz (worst slack -0.13 ns @ 3.66 ns period)
clk_fft     1923.08 MHz (worst slack 7.48 ns @ 8.0 ns period)
clk_spi     386.1 MHz (worst slack -1.59 ns @ 1.0 ns period)
clk_fp      1923.08 MHz (worst slack 3.48 ns @ 4.0 ns period)
Power:      0.00798 W
```

Figure 2: Baseline synthesis and STA results

5.3 The Iterative Optimization Loop

With the baseline established, the framework enters its core loop, repeating until either the design converges with no violations remaining or `MAX_ITERATIONS` is reached. Each pass through this loop targets exactly one violation and produces exactly one accept or reject decision.

5.3.1 Violation Selection

Each iteration begins by selecting the worst violation from the most recent timing report using `report_parser.py`. This selection deliberately **excludes** two categories of paths:

1. **Reset Recovery and Removal Paths:** These paths are excluded from the worst violation when the keywords `recovery` or `removal` are detected in the timing reports, and the corresponding path is captured separately.
2. **CDC Paths:** CDC exclusion uses a direct comparison of each path’s actual clock names, where `(src_clk != dest_clk)`, against `CLOCK_FAMILY_MAP` in `config.py`. This ensures that a gap in the SDC’s own clock grouping cannot allow a real CDC hazard to slip through undetected.

Note: These paths are usually set as false paths in `constraints.sdc`, but these features provide a two-layer protection mechanism to prevent the LLM from incorrectly attempting to optimize an asynchronous path.

The selected violation’s startpoint and endpoint are then resolved from raw netlist instance paths back into the actual RTL source using `module_extractor.py`. This uses a “**regex based parser**” which can scan and extract the required module and the particular instance within the module for both the startpoint and endpoint of the worst violating path.

This process walks through the instantiation hierarchy and tags each resolved module as either a **leaf** (the specific module actually containing the violating logic), or an **ancestor** (wrapper through which the path merely passes). These modules are then provided to `prompt_builder.py` with their respective classifications, so the LLM is guided toward the module that is actually the target for the fix.

```
=====
UNRESOLVED - EXCLUDED FROM THE AUTO-FIX LOOP
=====

Recovery/removal (async reset) violations: 1
  These check reset DEASSERTION timing, not data-path setup/hold.
  None of the 6 RTL optimization techniques apply - the correct
  fix is structural (reset synchronizer / adjusted reset release).
  NOT attempted by this framework; needs manual attention.
    recovery    slack    -1.160 ns    u_rst_sync_spi/_0_ -> u_spi/_1412_
```

Figure 3: Excluding unresolved CDC/Reset paths from optimization

5.3.2 Violation Classification

Here, `prompt_builder.py`’s heuristic classifier evaluates the violation’s stage by stage delay and fanout data against **six independent checks** to generate a prompt: **pipelining, logic restructuring, retiming, register balancing, FSM re-encoding, and resource duplication**. Any subset of these checks may be triggered. These are offered to the LLM as optional starting points rather than constraints, since the classifier is a structural heuristic and not a guarantee of the correct fix. The diagram below represents the basis for the classification of each violation:

(`stage_count`) = Number of gates/flops a particular path passes through before it is captured
 (`max_stage_delay`) = Ratio of the Maximum delay a stage and the Total delay in that path

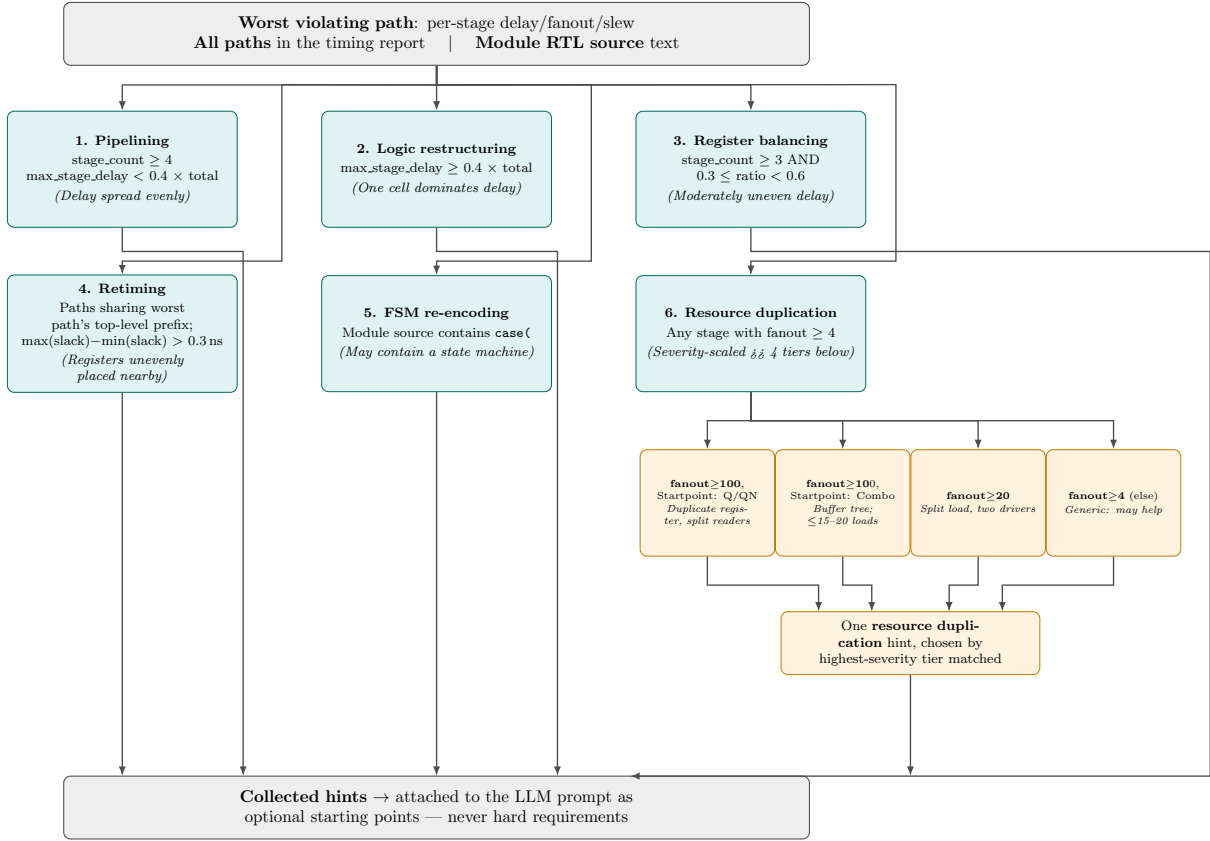


Figure 4: Heuristic classification in `classify_violation()`

A few internal features of how a prompt is generated are as follows:

1. **Setup and Hold Violations:** Setup and hold violations have their own different styles of prompts. The setup based prompt is detailed and depends on the type of violation and how the framework has classified the violation. The hold prompt asks the LLM to be more conservative by suggesting that it should not speed up or remove logic unnecessarily, and instead **add minimum delay to the hold violating paths** without affecting other paths.
2. **Specified Response Format:** The prompt enforces the LLM to send back three things in the specified format:
 - (a) One module of code, a strict requirement that only one module must be returned.
 - (b) A one line description of the changes or optimization performed in the updated RTL.
 - (c) The number of pipeline stages added, if any.
 - (d) Stretched ports: The list of output ports in the module which will not be delayed by the added pipeline stages (EQY).

```

--- gemini's self-report ---
Fix description      : Converted single-cycle restoring array divider into a 32-cycle iterative divider to break the long combinational path.
Pipeline stages added : 32
Stretched ports      : none reported
  
```

Figure 5: Markdown log of LLM's response captured (PicoRV SOC example)

3. Every attempt of the prompt suggests optimization techniques to the LLM, such as pipelining, retiming, and others. If an attempt fails, that particular technique is considered re-

jected, while the remaining techniques are suggested in the next attempt. This mechanism is used only for internal bookkeeping and is never enforced as a strict rule in the prompt.

4. **Alternative Prompt:** When one of the patches fails due to a formal verification failure or no improvement in the slack, then the next prompt will contain the Base prompt + The previous rejection reason to avoid the LLM from making the same error.
5. **Retry Prompt:** If the `orchestrator.py` reports that the patch from the LLM has syntax errors, the LLM is fed back the broken code + the `iverilog` error for it to rectify the error.

5.3.3 LLM-Driven Patch Generation

The constructed prompt is sent to whichever provider is configured, Claude, Gemini, or GPT, through `llm_client.py`'s uniform interface. The response is parsed for three things: the actual Verilog patch, a self-reported description of the fix, and critically, a self-reported pipeline stages added count, the number of clock cycles of latency the patch introduced, if any with the Stretched ports explicitly mentioned. This last value directly determines how formal verification is later performed (Section 5.3.5). `llm_client.py` has a “**self contained retry loop**” which performs retries directly with the LLM and only reports to the user/`orchestrator.py` when all the retries fail or there is a timeout.

The `orchestrator.py` sees only one of the 2 following things from the `llm_client.py`:

1. Complete version of {RTL code, One line description, Pipeline stages, Stretched ports}
2. `LMError` (No code/output given by the LLM): This will lead to rejection of the particular iteration and proceeds to the next one.

5.3.4 Patch Validation (Icarus Verilog)

Before the patch is trusted at all, `patch_applier.py` enforces that it modifies exactly one module (Counts every module in the patch, if not equal to 1, patch error), locates the target modules among the RTL files and surgically replaces the particular module inside the file. `patch_applier.py` also incorporates multiple ways of exhaustive testing to check if the patch was correctly replaced. Once verified, writes it into a new, versioned iteration directory, leaving every prior iteration's RTL untouched.

The patch then passes through `syntax_check.py`, which runs **Icarus Verilog** in parse only mode. A syntax failure does not end the iteration outright: the framework rebuilds the prompt with the LLM's own broken code and the exact compiler error attached, giving it a bounded number of chances, `MAX_SYNTAX_RETRIES`, to self correct before the iteration is abandoned. (As mentioned in Section 5.3.2)

```
=====
ITERATION 1
=====
Worst violation: u_rst_sync_spi/_0_ -> u_spi/init_USR/_204_
check type : setup
path group : clk_spi_div
slack      : -1.59 ns
stages     : 3
Resolved to module(s): reset_sync, spi, USR
Suggested technique(s): logic restructuring, retiming, register balancing, resource duplication
Requesting patch from gemini...
Direct use of automatic function calling (AFC) in Models.generate_content is not recommended. Instead, we recommend to use AFC in Chat.send_message. Similarly, direct use of AFC in Models.generate_content_stream is not recommended. Instead, we recommend to use AFC in Chat.send_message_stream.
Failed to obtain a usable patch: Patch rejected: Patch contains 2 module declarations (to, spi), expected exactly 1. Only one module may be modified per patch - resubmit with just the single module that needs the fix.
SUMMARY: patch failed before synthesis | REJECT
=====
```

(a) Patch contains 2 Modules

```
=====
ITERATION 2
=====
Worst violation: u_accel_top/u_accel/u_div/_9030_ -> u_accel_top/u_accel/u_div/_9095_
check type : setup
path group : clk_div
slack      : -51.58 ns
stages     : 412
Resolved to module(s): picorv32_accel_top, accel_top, int_div
Suggested technique(s): pipelining, retiming, resource duplication
Previously attempted (rejected): pipelining
Requesting patch from gemini...
Syntax check failed (attempt 1/4):
verilog.iterations/iter_2/int_div.v:26: syntax error
verilog.iterations/iter_2/int_div.v:26: error: Malformed statement
Re-prompting the LLM with the compiler error...
Patch applied to module 'int_div' -> verilog.iterations/iter_2
Syntax check passed (1 retry/retries used).
gemini did not report a fix description (missing marker).
Re-synthesizing and re-running STA...
```

(b) Patch contains syntax error

Figure 6: Representative Iteration failures handled by the Closed loop

5.3.5 Re evaluation

Once a syntactically valid patch exists, the framework re runs the same synthesis and STA steps from Section 5.2 but now scoped to just the patched module, using its correct per domain clock target from `MODULE_CLOCK_MAP` (`config.py`). The resulting updated timing, area, and power numbers are parsed the same way as the baseline, thereby giving a direct and apples to apples before and after comparison for this specific violation.

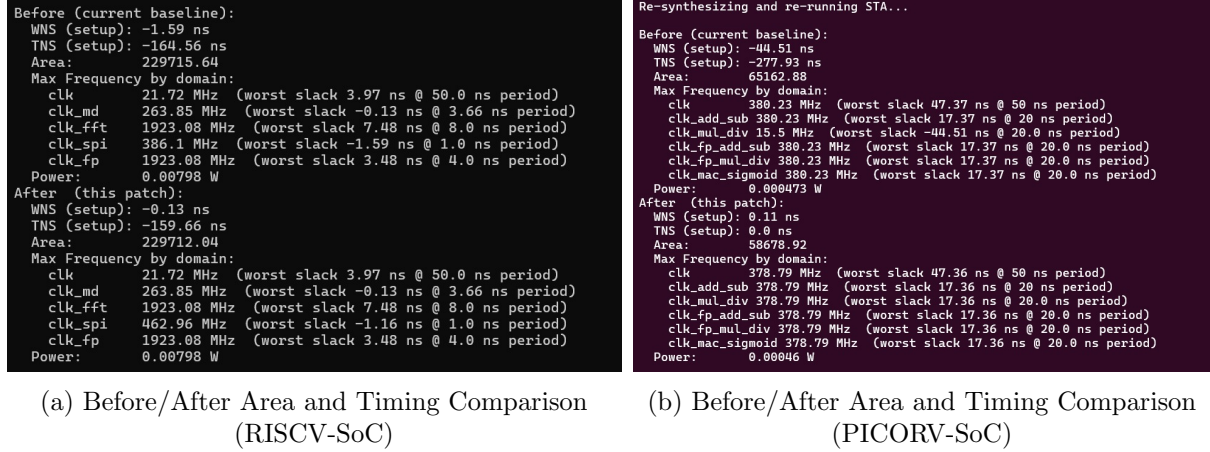


Figure 7: Timing and area comparison before/after applying an accepted RTL patch

5.3.6 Formal Safety Verification (EQY/SymbiYosys)

Every patch that reaches this point is proven functionally equivalent to the pre patch RTL via EQY (`formal_check.py`) where this is the framework’s non negotiable safety gate. `formal_check.py` It works as a **formal script automator** as well, similar to synthesis, where it can dynamically generate formal verification EQY scripts for any 2 input designs (User need not create these scripts before hand, done by the framework)

Latency Aware Equivalence Checking: When the LLM reports that a patch introduces pipeline stages, the framework generates a `gold.wrapper.v` around the original baseline module that delays its outputs by the reported number of cycles, so a pipelined patch is compared against the correct cycle offset instead of being incorrectly flagged as non-equivalent. The patched module itself is not wrapped. Ports identified as “Stretched Ports” are not added any pipeline delay in the gold wrapper are OR combined across all intermediate delay stages in the `gold.wrapper.v`, keeping the signal asserted over the extended pipeline window rather than only at the final delayed cycle. The EQY and SymbiYosys tools have been used in the following manner:

1. **SAT solver:** First attempts formal equivalence check for the specified timeout. If it times out, the flow proceeds to the SBY based verification.
2. **SBY:** Takes over after SAT timeout and performs formal verification using SymbiYosys with the `smtbmc` engine and Bitwuzla solver.

```

# Formal Equivalence Verification Report
Run: 20260913195943101de9

## Overall Design Verification
- Top module checked: `soc_top`
- Result: **PASS** - the final design is formally equivalent to the original baseline.
- Log: `formal/overall/eqy_log.txt`

## Per-Module Verification (Per Iteration)
**2/2 per-module checks passed formally.**

| Iter | Module | Result | Log |
|---|---|---|---|
| 1 | USR | PASS | `formal/iter_1/eqy_log.txt` |
| 2 | Multiplier | PASS | `formal/iter_2/eqy_log.txt` |

```

Figure 8: Dedicated Formal Verification Report Summary

5.3.7 Accept / Reject Decision

With updated timing and a formal verdict in hand, the orchestrator’s decision matrix determines the outcome. A patch is accepted only if it is formally equivalent and shows genuine slack improvement above `SLACK_IMPROVEMENT_THRESHOLD` (`config.py`). If formal verification fails outright with no improvement, it is rejected automatically. The genuinely ambiguous case formal verification fails, but slack improves anyway, which can legitimately happen with a correctly latency shifted patch is not resolved by guessing:

1. If `INTERACTIVE_MODE = 1`: Decision is deferred to the user, asks the user before moving to the next iteration.
2. If `INTERACTIVE_MODE = 0`: Defaults to a safe automatic rejection and moves to the next iteration without user interference

Rejected patches are not simply discarded. On the next attempt at that same violation, `prompt_builder.py` builds an alternate prompt that states the specific rejection reason and explicitly excludes every technique already tried, so the LLM is steered toward genuinely different approaches rather than minor variations of a known failure (Section 5.3.2).

5.3.8 Iteration Logging

Regardless of outcome, every iteration the violation, technique attempted, before and after metrics, formal verdict, and final decision is recorded to a structured log by `logger.py`. This log is the single source of truth every later report and the dashboard are built from.

5.4 Whole Design Formal Sign off

Module level proofs alone do not guarantee the composed design is still correct. Before physical implementation is considered, the framework runs one final, harder EQY check the original baseline RTL vs. the complete final RTL, at the full top level module with a larger timeout budget (`OVERALL_EQY_TIMEOUT`, `OVERALL_SAT_TIMEOUT`), since proving equivalence across a whole SoC is harder than for one module.

A timeout or tooling error is never treated as a pass. Even with `PNR_ENABLED = 1`, if this check fails, Physical Design is skipped entirely **an unproven design is never sent to place and route**. The verdict and log path become the headline of the **dedicated Formal Equivalence Verification Report**, which also lists every per module check with pass fail counts and log

references, one coherent artifact for this deliverable.

5.5 Physical Design (OpenROAD)

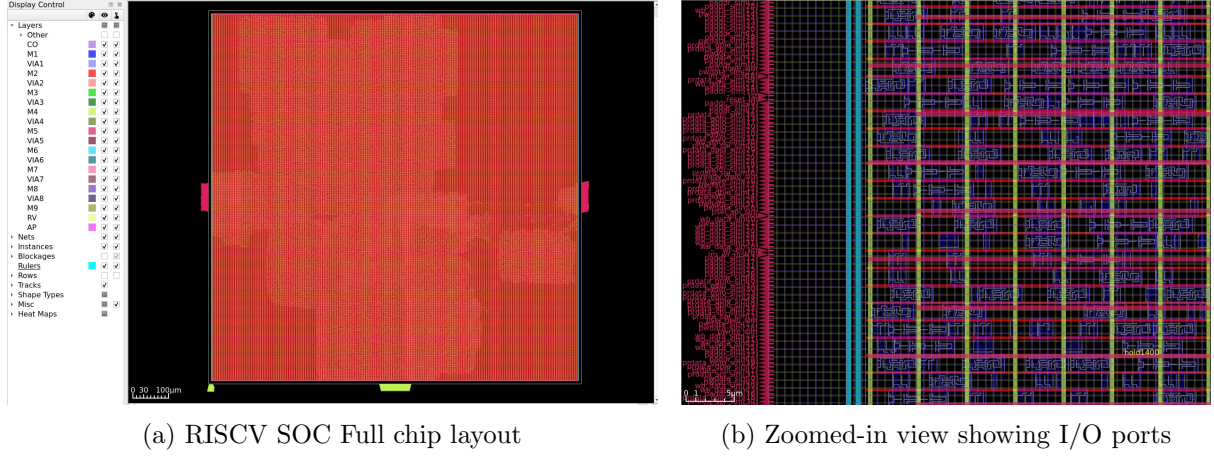


Figure 9: Chip layout from OpenROAD: full view and zoomed I/O detail

Physical Design is the final, optional stage of the pipeline a single, one shot run, not something repeated per iteration. It only executes if two conditions both hold: `PNR_ENABLED` is set (Section 5.4), and the whole design formal sign off from Section passed. **A design that has not been proven equivalent to its original baseline is never sent to place and route, regardless of configuration.**

Execution: `pnr_runner.py` mirrors the same isolated workspace pattern used throughout the framework: it copies the final accepted RTL into a dedicated **PnR** workspace and runs the project's hand written `pnr.tcl` script through **OpenROAD**. Once post route metrics are extracted, the framework automatically compares them against the synthesis stage numbers for the same final design.

Automatic Timing Flip Detection: Any clock domain whose met or violated status changes between the two stages. This is detected from each domain's actual worst slack value. Sign flips exist precisely because synthesis stage timing has no real wire parasitics a path that looks comfortably met before layout can genuinely fail once real routing delay is accounted for, and the reverse a path recovering slack thanks to the router's own optimization is possible too, though less common.

```
=====
PHYSICAL DESIGN
=====
Physical design (PnR): running OpenROAD sign-off on the final accepted baseline...
PnR complete. Reports: pnr/pnr_runs/final/reports

PPA Comparison: Synthesis-stage vs. Post-Route
WNS (setup): 0.02 -> 0.0 ns
TNS (setup): -164.62 -> 0.0 ns
Area: 229541.4 -> 264357.0
Power: 0.008 -> 0.009 W
Max Frequency by domain:
clk 21.7 -> 21.7 MHz
clk_fft 1923.08 -> 125.0 MHz
clk_fp 1923.08 -> 460.83 MHz
clk_md 274.73 -> 487.8 MHz
clk_spi 456.62 -> 2173.91 MHz
```

Figure 10: Physical Design PPA Comparison (Markdown Version)

6 Reporting and Dashboard

The final stage of every run turns the structured log accumulated throughout into artifacts a human can actually read and evaluate both as documents and as an interactive visual summary.

6.1 Structured Reports (`logger.py`)

The iteration log (`run.jsonl`) is the single source of truth with one JSON record per iteration, capturing the violation, technique attempted, before and after timing and power, the formal verdict, and the final decision. Every other report and the dashboard itself are built entirely from this one file. The **Markdown report** presents this as a readable summary with key components like iteration count, PPA comparisons, violations and LLM fixes, etc

6.2 Dedicated Formal Equivalence Verification Report

Exists as its own artifact, separate from the general Markdown report, directly satisfying the requirement for a standalone formal verification deliverable. It has the

1. Whole design sign off verdict pass, fail, timeout, or disabled (with log path)
2. Per module table listing every individual formal check run during the loop, with a pass fail count summary and a direct log reference for each entry

This exists specifically because formal results would otherwise be scattered across raw per iteration EQY logs with no single place pointing to the overall verification story.

6.3 The Offline HTML Dashboard (`generate_demo.py`)

The dashboard is a self contained HTML file with no external dependencies or server requirements. It opens directly in any browser, making it reliable for offline demos.

It presents, in order:

- Summary table comparing baseline, synthesis, and post route metrics, with improvements calculated
- Timing and area charts showing WNS and area across all iterations
- Decision breakdown showing accepted, rejected, and pending iterations
- Per iteration details including violations, technique hints, LLM fix description, formal result, and decision reasoning
- Unresolved exclusions highlighting issues that still require manual attention

Figure 11 shows this summary as rendered for the benchmark design run, with every metric's improvement computed automatically.

NEBULA - GenAI RTL Timing Optimization
Run: 20260913_195943_101de9 | 2 iteration(s) | 2026-09-13T20:07:20.774824

Baseline -- Final Result

Metric	Baseline Synthesis	Optimised Synthesis	Δ (Optimized)	% Improvement (Optimized)	PNR Results
WNS (setup)	-1.59 ns	0.02 ns	▲ 1.61 ns	—	0.00 ns
TNS (setup)	-164.56 ns	-164.62 ns	▼ -0.06 ns	—	0.00 ns
Area	229715.6 μm^2	229541.4 μm^2	▼ -174.2 μm^2	0.1% better	264357.0 μm^2
Power	0.00798 W	0.00794 W	▼ -0.00004 W	0.5% better	0.00879 W

FREQUENCY BY DOMAIN

Domain	Baseline Synthesis	Optimised Synthesis	Δ (Optimized)	% Improvement (Optimized)	PNR Results
clk	21.72 MHz	21.70 MHz	▼ -0.02 MHz	0.1% worse	21.70 MHz
clk_fft	1923.08 MHz	1923.08 MHz	— 0.00 MHz	0.0% change	125.00 MHz
clk_fp	1923.08 MHz	1923.08 MHz	— 0.00 MHz	0.0% change	460.83 MHz
clk_md	263.85 MHz	274.73 MHz	▲ 10.87 MHz	4.1% better	487.80 MHz
clk_spi	386.10 MHz	456.62 MHz	▲ 70.52 MHz	18.3% better	2173.91 MHz

▲ Timing sign flip(s) between synthesis and post-route:
• clk_spi: VIOLATED at synthesis-stage STA, but MET post-route - less common; possibly OpenROAD's own CTS/routing optimization recovered slack the synthesis-stage estimate didn't account for.

Figure 11: Snapshot of the PPA summary (Example from the benchmark design - RISCv-SoC)

Convergence is rarely monotonic in practice; Figure 12 captures a brief slack regression before the design ultimately stabilizes.



Figure 12: Snapshot of Timing/Area graph (Example from PicoRV-SoC)

Figure 13 makes every accept/reject call traceable, including rejections that followed an initially promising slack change.

Decision Breakdown

user_accept: 4 | reject: 4

Iteration Detail (click a row to expand)

#1	int_div	-29.62 → -28.79 ns	ACCEPTED (user)
#2	fp_add	-28.79 → -24.97 ns	ACCEPTED (user)
#3	fp_sub	-24.97 → -29.48 ns	REJECTED
#4	fp_sub	-24.97 → -17.74 ns	ACCEPTED (user)
#5	fp_add	-17.74 → -16.88 ns	ACCEPTED (user)
#6	fp_sub	-16.88 → -10.54 ns	REJECTED
#7	fp_sub	-16.88 → -17.92 ns	REJECTED
#8	fp_sub	-16.88 → -9.65 ns	REJECTED

Figure 13: Snapshot of per-iteration accept/reject decision log (Example from PicoRV-SoC)

7 Findings During Exhaustive Testing of the LLM

LLM Comparison

Metric	Claude	Gemini	OpenAI (GPT)
Model used	claude-sonnet4.6	gemini-3.6-flash	gpt-5.5
SDK	anthropic	google-genai	openai ($\geq 2.1.0$)
Reasoning/Thinking Model	Supports extended thinking (Not enabled)	Default Reasoning (Confirmed from truncated patches)	Default Reasoning (Documented Risk)
Rate limit (free tier)	No free tier (paid only)	20 responses/day	3 RPM / 50 RPD / 10k TPM (very insufficient)
LLM efficiency	Most efficient for self-reporting	Basic, decent enough	Sometimes returns wrong patch

Table 2: Comparison of the three LLM providers used in this project.

Other Key Findings

1. **Shadow-register bug:** LLM pre-computed values into a synced shadow register; EQY cannot see cross-register logic, flagged it non-equivalent despite correct behavior. Fixed by banning the pattern in-prompt.
2. **Unreliable self-reported metadata:** LLM often omits `FIX_DESCRIPTION/PIPELINE_STAGES_ADDED`. Defaults to 0 stages, forcing the strictest formal check.
3. **Module-count non-compliance:** LLM sometimes returns 0 or 2+ modules despite explicit instruction. Enforced by hard rejection at patch validation, not prompt wording alone.
4. **Hold fixes need a narrower prompt:** Setup-style hints risk speeding up a path, the wrong direction for hold. Required a separate, conservative prompt.
5. **Reasoning tokens silently starve visible output:** Gemini 3-series spends `max_output_tokens` on internal reasoning first; large modules (`controller.v`, $\sim 12,700$ tokens to echo unchanged) produced truncated, malformed patches. `thinking_budget=0` rejected by the API. Mitigated via `max_output_tokens=32768`; unresolved on very large modules.
6. **Correct bug fixes fail equivalence by construction:** A patch fixing a real latent bug (reset loop bound vs. array size) was correctly flagged non-equivalent by EQY, since fixing a bug is a behavioral change from the buggy baseline. EQY cannot distinguish “wrong” from “now correct”, requiring a human in the loop.
7. **Fanout-dominated delay ignores timing budget:** Identical arrival time at 30ns vs 50ns targets, to two decimals. Past a fanout ceiling, driver-strength selection cannot improve regardless of budget, requiring RTL restructuring rather than a bigger target.
8. **Latency fixes can stretch a signal’s duration, not just shift its timing:** Converting a combinational op to iterative widened a status pulse (`busy`) from 1 to 32 cycles; a per-cycle delay only replays it later, not wider. The fix broke on back-to-back requests, producing a state neither design exhibits, confirmed by simulation, requires human review.

8 Conclusion

Nebula delivers a complete, working closed-loop framework taking a real multi-clock SoC from timing violations through GenAI-driven optimization, formal verification, and physical implementation, with measured improvements in WNS, TNS, area, power, and frequency, every accepted patch backed by proof.

All required deliverables were met: automated report parsing, LLM-driven patch generation, per-patch formal equivalence verification, a dedicated Formal Verification Report, PPA comparison, and a working end-to-end pipeline.

Beyond that, the project added: full Physical Design integration with post-route PPA and sign-flip detection, a whole-design formal sign-off gate before PnR, support for three interchangeable LLM providers, validation across three structurally different designs, and a self-contained offline HTML dashboard.

The project also pushed far enough to find real, honestly-documented limits of automated formal verification, cases needing human judgment rather than a cleverer workaround. **Nebula** is a working, extensively verified foundation, with clear next steps already identified.